# Reliability in Spring Boot on AWS

A practical guide to distinguishing reliability from availability and scalability, designing for failure, and explaining the approach in a Senior Engineering Manager interview.

## 1. What is reliability?

**Reliability is the ability of a system to perform its intended function correctly and consistently over time, including when failures occur.**

- **Availability:** Can users access the application?
- **Scalability:** Can the application handle growth in users, traffic, or data?
- **Reliability:** Does it consistently produce correct results, protect data, and recover safely?

For example, an API may return HTTP 200 and still be unreliable if it creates duplicate payments, loses an order, returns stale data, or silently drops a Kafka message.

## 2. How to ensure reliability

### 1. Fault tolerance

- Deploy across multiple Availability Zones using an Application Load Balancer and multiple EC2 instances or ECS/EKS tasks.
- Use health checks to remove unhealthy instances from traffic.
- Apply timeouts, retries with exponential backoff, and circuit breakers (for example, Resilience4j).
- Use bulkheads to prevent one failing dependency from exhausting all application resources.

### 2. Data integrity and consistency

- Use database transactions and appropriate isolation levels.
- Use idempotency keys to prevent duplicate processing when clients retry requests.
- Use RDS Multi-AZ for database high availability.
- For database-plus-Kafka workflows, consider the transactional outbox pattern to avoid losing events between database commits and event publication.

### 3. Monitoring and observability

- Monitor latency, error rates, throughput, and resource saturation.
- Use CloudWatch metrics, alarms, and logs.
- Use distributed tracing, such as AWS X-Ray or OpenTelemetry.
- Track business-level failures too, such as orders stuck in processing or messages accumulating in a queue.

### 4. Testing resilience

- Use unit, integration, and contract tests.
- Run load, stress, and soak tests.
- Use controlled failure injection or chaos testing.
- Test database unavailability, Kafka outages, slow downstream APIs, and instance termination.

### 5. Recovery and disaster preparedness

- Automate backups and regularly test restores.
- Define RTO (Recovery Time Objective) and RPO (Recovery Point Objective).
- Use Multi-AZ for availability-zone failures; consider cross-region disaster recovery when business requirements justify it.
- Maintain runbooks and regularly test recovery procedures.

### 6. Safe deployments and configuration

- Use rolling, canary, or blue-green deployments.

- Apply automated smoke tests and rollback criteria.
- Manage configuration and secrets securely.
- Use backward-compatible database migrations and feature flags where appropriate.

## 3. Example: an order-processing service

Suppose a Spring Boot service processes customer orders, calls a payment service, and publishes an event to Kafka. Reliability means designing for these failure cases:

| Failure scenario | Reliability mechanism |
|---|---|
| One application instance crashes | Load balancer routes traffic to healthy instances. |
| Payment service becomes slow | Timeout and circuit breaker prevent resource exhaustion. |
| Client retries the same order | Idempotency prevents duplicate orders or charges. |
| Kafka is temporarily unavailable | Durable event storage, retry, and recovery mechanisms. |
| Database primary fails | RDS Multi-AZ failover with appropriate retry handling. |
| A new release introduces defects | Canary deployment, monitoring, and rollback. |
| Data is accidentally corrupted | Backups, restore procedures, and data-integrity controls. |

**Key principle:** Reliability is not about preventing every failure. It is about limiting the impact of failures and ensuring the system behaves correctly when they happen.

## 4. How to measure reliability

| Metric | What it tells you |
|---|---|
| Availability / SLO | Percentage of eligible requests served successfully. |
| Error rate | Percentage of requests failing. |
| p95 / p99 latency | Whether response times remain within acceptable limits. |
| MTTR | How quickly service is restored after an incident. |
| Data loss / duplicate rate | Whether business transactions remain correct. |
| RTO / RPO | Whether disaster recovery meets agreed recovery-time and data-loss objectives. |

Example: a 99.9% availability SLO over a 30-day month allows approximately 43.2 minutes of downtime when calculated using time-based availability. A request-based SLO instead measures the permitted share of failed or bad requests.

## 5. Interview-ready answer

> *“I look at reliability as the ability of an application to consistently deliver correct business outcomes, even when infrastructure, dependencies, or individual components fail.*
>
> *For a Spring Boot application on AWS, I would approach reliability across multiple dimensions. At the infrastructure level, I would use multi-AZ deployments, load balancing, and health checks. At the application level, I would implement timeouts, controlled retries, circuit breakers, and idempotency. At the data layer, I would focus on transaction integrity, database failover, backups, and recovery.*
>
> *I would also establish observability using metrics, logs, and distributed tracing, define availability and latency SLOs, and validate resilience through failure testing. Finally, I would use safe deployment strategies and regularly test disaster recovery.*
>
> *From a leadership perspective, I would make reliability measurable through SLOs, error budgets, incident trends, MTTR, and recurring problem analysis. The objective is not simply to keep the application running, but to ensure it delivers correct outcomes and recovers predictably when failures occur.”*

## Takeaway

**Availability** tells you whether the system is accessible. **Reliability** tells you whether it can be trusted to work correctly and consistently over time. A highly available system can still be unreliable if it returns incorrect results or loses data.